

Expressions lambda

Une expression lambda représente une fonction:

<code>lambda</code>	<code>x</code>	<code>:</code>	<code>f(x)</code>
Un fonction qui	prend x	et retourne	f(x)

```
def square(x):  
    return x*x  
>>> square(5)  
25
```

```
square = lambda x : x*x  
>>> square(5)  
25
```

```
>>> (lambda x : x*x)(5)  
25
```

Expressions lambda

```
def cube(k):  
    return k**3
```

```
def sum_cubes(n):  
    return summation(n,1,cube)
```

```
def summation(n,m,f):  
    total = 0  
    for k in range(0,n+1,m) :  
        total += f(k)  
    return total
```

```
def sum_cubes(n):  
    def cube(k):  
        return k**3  
    return summation(n,1,cube)
```

Cachez la fonction auxiliaire dans une autre

```
def sum_cubes(n):  
    cube = lambda k : k**3  
    return summation(n,1,cube)
```

plus court avec expression lambda

```
def sum_cubes(n):  
    return summation(n,1, lambda k : k**3)
```

encore plus court

$$\sum_{k=0}^n k^3$$

Fonction comme valeur retournée

```
def cube(x) :  
    return x**3  
  
def double(x) :  
    return 2*x  
  
def compose(f, g):  
    def h(x):  
        return f(g(x))  
    return h
```

```
>>> compose(double, cube)(2)
```

```
16
```

```
>>> compose(cube, double)(2)
```

```
64
```

Avec expressions lambda

```
cube = lambda x : x**3
```

```
double = lambda x : 2*x
```

```
def compose(f, g):  
    return lambda x : f(g(x))
```

```
>>> compose(double, cube)
```

```
<function compose ... >
```

```
>>> lambda x : double(cube(x))
```

```
<function <lambda> ... >
```

```
>>> compose(double, cube)(2)
```

```
16
```

```
>>> compose(cube, double)(2)
```

```
64
```